pillar-UDP Redundancy: Header Format, the Loss/Bandwidth Trade, and Adaptive Congestion Safety

A companion to "Clustered Multipath Congestion Avoidance and Redundant Delivery over Lossy Networks"

pillar engineering

September 9, 2026

Abstract

The primary pillar-UDP paper proves the protocol's *qualitative* delivery guarantees—exactly-once processing, route-around liveness, and bounded fan-out—and defers the *quantitative* design to this companion. We supply three things. First, the analytic loss/bandwidth trade that sets the per-connection *redundancy allowance*: a closed form for the minimum copy count $W^\star = \lceil \log_p \varepsilon \rceil$ in the small-message regime and a binomial residual for the erasure-coded bulk regime, with the overhead each implies. Second, a correlated-loss model—an exchangeable Pólya/Beta-binomial mixture with a single correlation parameter ρ—that quantifies exactly how much shared loss erodes redundancy, showing that the whole benefit of pillar-UDP rides on driving ρ toward zero by AS-diverse dispersion, and turning the "paths are independent" claim from an assumption into a measurable quantity with a stated methodology. Third, the concrete header wire format and an adaptive control loop that sets the allowance from measured loss while remaining *congestion-safe*: a delay-gradient discriminator prevents redundancy from being added in response to congestion (as opposed to link loss), and a marginal-gain rule plus an aggregate-egress bound cap fan-out so the adaptive allowance cannot itself induce collapse. Every numeric result here is exact arithmetic from the closed forms; the one empirical question—the real-world distribution of ρ—is framed as a measurement study, not filled with invented data.

1 Introduction

pillar-UDP delivers a message by spraying a *bounded* number of redundant, content-addressed datagrams across a topologically dispersed set of nodes and letting the receiver deduplicate by CID; loss is absorbed by redundancy and congestion is avoided by finding a good route rather than by throttling a window. The primary paper [1] model-checks that this delivers exactly once and always routes around lossy links whenever the cell stays reachable by some dispersed path. What it deliberately leaves open is every *number*: how many copies, how much bandwidth that costs, how to encode the choice on the wire, how to adapt it to a changing link, and how to keep the adaptation from becoming a self-inflicted congestion source. This paper closes those gaps.

The engineering tension is a single trade. Redundancy buys reliability super-linearly (p^W falls fast in W) but costs bandwidth linearly (fan-out W). Setting the redundancy allowance is therefore an optimisation with three inputs—the measured per-path loss p, the connection's target residual ε, and the correlation ρ between the paths the copies take—and one hard constraint: the aggregate traffic a cell emits must never exceed what its egress can carry, or the "cure" becomes the disease. We treat each in turn.

2 System model

A *message* is a small, signed, content-addressed unit identified by its CID. A *connection* carries a stream of messages between a client and a cell or between two cells and declares a target residual loss ε (the probability a message is ultimately undelivered that the application will tolerate). A *path* is one dispersed source/route a copy takes; the sender controls how many paths it uses. We write p for the per-path loss probability over the averaging window and treat per-path losses as a marginal-p Bernoulli whose *dependence* across paths is captured by a correlation parameter $\rho \in [0, 1]$ (§4); $\rho = 0$ is fully independent, $\rho = 1$ is one shared fate. Two redundancy shapes share this substrate: *copy redundancy* (a whole message sent W times, for small messages) and *erasure redundancy* (K data + M parity shards, MDS target reconstructs from any K, for bulk). Both are bounded by the same per-connection allowance field (§8).

3 The loss/bandwidth trade: small messages

Under independent per-path loss, a message sent as W copies is undelivered only if all W drop:

$$P_{\text{fail}}(W, p) = p^{\,W}. \tag{1}$$

The bandwidth cost is a fan-out of W (overhead $W - 1$ redundant copies per delivered message). Fixing the

p	ε=10^{-3}	ε=10^{-6}	ε=10^{-9}
0.01	2	3	5
0.05	3	5	7
0.10	3	6	9
0.20	5	9	13
0.30	6	12	18
0.50	10	20	30

Table 1: Minimum copy allowance $W^\star = \lceil \log_p \varepsilon \rceil$ (Eq. 2). Cheap on good paths; grows only as the path degrades.

connection's target ε and solving $p^W \le \varepsilon$ gives the minimum allowance in closed form:

$$W^\star(p, \varepsilon) = \left\lceil \frac{\ln \varepsilon}{\ln p} \right\rceil = \lceil \log_p \varepsilon \rceil . \tag{2}$$

This is the value the sender computes each averaging window and writes into the allowance field. It scales *down* to $W = 1$ as $p \to 0$ (a clean path costs nothing beyond the message itself) and *up* as $p \to 1$, which is precisely the adaptive posture the primary paper requires and the opposite of a fixed redundancy multiplier. Table 1 tabulates $W^\star$; it is exact arithmetic from (2). Even a 30%-loss path needs only W=12 copies for a one-in-a-million residual, and a 10% path needs 6.

Efficiency. Delivered-message yield per unit bandwidth is $(1 - p^W)/W$: it is maximised at W=1 and decreases monotonically, so redundancy is never "free throughput"—it is reliability purchased with bandwidth. The correct operating point is therefore not an efficiency optimum but the *smallest* W meeting ε, i.e. $W^\star$; spending more wastes bandwidth and spending less misses the target. This is why the allowance is a hard per-connection budget and why the controller of §6 tracks $W^\star$ rather than hunting a throughput maximum.

Latency. Because the W copies travel in parallel rather than as RTT-spaced retransmissions, delivery latency is the *minimum* arrival time over the surviving copies, not a retransmit timeout after a loss. Redundancy thus improves tail latency on lossy paths at the same time as it improves delivery—the property single-path protocols cannot get, since their only recovery is to wait an RTT and resend.

4 Correlated loss and the independence assumption

Equation (1) assumes the W paths fail independently. That assumption is the load-bearing one: if the copies share a bottleneck, redundancy buys far less. We make the dependence explicit with an exchangeable mixture. Let the common "network weather" over the copies be

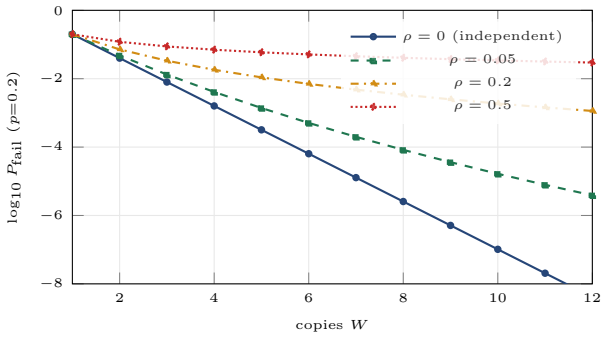

Figure 1: Residual loss vs. copy count under correlated loss (Eq. 3, p=0.2). Independence (ρ=0) is a straight log decline; correlation flattens the curve, and past modest ρ extra copies stop helping. Redundancy's value is entirely a function of how uncorrelated the dispersed paths are.

a latent loss level q with mean $\mathbb{E}[q] = p$ drawn from a Beta(a, b) distribution; conditioned on q the W copies drop independently. Then

$$P_{\text{fail}}(W, p, \rho) = \mathbb{E}[q^W] = \prod_{i=0}^{W-1} \frac{p\varphi + i}{\varphi + i}, \qquad \varphi = \frac{1-\rho}{\rho}, \tag{3}$$

where $\rho = 1/(a{+}b{+}1)$ is the pairwise loss correlation and $\varphi = a{+}b$. This is a Pólya/Beta-binomial urn and it interpolates exactly between the two extremes: as $\rho \to 0$ ($\varphi \to \infty$), (3) $\to p^W$, the independent case; as $\rho \to 1$ ($\varphi \to 0$), it $\to p$—i.e. W copies on one shared fate are worth a single copy. By Jensen's inequality $\mathbb{E}[q^W] \ge (\mathbb{E}q)^W = p^W$, so *any* positive correlation strictly erodes redundancy, and the erosion compounds with W.

Figure 1 plots (3) for p=0.2. The message is stark: at ρ=0 ten copies reach 10^{-7}; at a modest ρ=0.2 the same ten copies stall near $10^{-2.7}$ and adding more barely helps; at ρ=0.5 redundancy is nearly worthless. *This is the analytic justification for dispersion.* pillar-ipam binds addresses to topology tiers and sites hold distinct prefixes, so "spray from a dispersed set of IPs" is exactly the operation that pushes the copies onto uncorrelated failure domains and drives $\rho \to 0$. Dispersion is not a nicety; Eq. (3) shows it is the difference between redundancy that works and redundancy that does not.

Making ρ measurable (methodology, not results). Equation (3) reduces the open empirical question to a single estimand: the pairwise loss correlation ρ between dispersed pillar paths, and its dependence on how much topology diversity separates them. We do *not* report a value here—measuring it requires a live multi-site deployment we treat as future work—but we fix the method so the number, once gathered, plugs straight into the model. (i) From concurrent per-path delivery logs across pairs of pillar-ipam prefixes at increasing topology distance (same rack, same site, same region,

cross-region), estimate the empirical joint drop rate and invert (3) for ρ at each distance. (ii) Fit the burst structure with a two-state Gilbert–Elliott channel [4, 5] per path and measure cross-path state correlation, which yields ρ directly. (iii) Feed the measured ρ(distance) back into $W^\star$ by replacing p in (2) with the correlation-inflated effective loss from (3). Until those measurements exist, the controller (§6) uses a conservative ρ prior and the safety envelope (§7) does not depend on the value.

5 Bulk transfer: erasure coding

For payloads larger than a datagram, copy redundancy is wasteful: sending a megabyte W times to survive loss pays an integer multiple. Bulk instead uses a *systematic erasure code*: K data shards plus M parity shards, $N = K + M$ total. The design target is a *maximum-distance-separable* (MDS) code—any K of the N shards reconstruct the payload—as provided by Reed–Solomon [6] or a fountain code such as RaptorQ [7]. Under independent per-shard loss p, an MDS code fails to reconstruct iff fewer than K shards arrive:

$$P_{\text{fail}}(K, M, p) = \sum_{i=0}^{K-1} \binom{N}{i} (1-p)^i p^{N-i}. \qquad (4)$$

The overhead is M/K, a *fractional* rate tunable independently of K, where copy redundancy pays a whole multiple. Figure 2 plots (4): at 20% loss, K=8, M=8 (overhead 1.0) reaches $10^{-2.8}$, and increasing the block to K=16, M=8 (overhead 0.5) does about as well because a larger block averages the loss. The minimum parity $M^\star(K, p, \varepsilon)$ meeting a target follows by the same solve as $W^\star$; Table 2 reports it for K=8, ε=10^{-6}. The mapping is the design rule: the single allowance field encodes a parity *rate* (§8) and the sender derives $M = \lceil \text{rate} \cdot K \rceil$ from it and the block size K.

Implementation status. The shipping `encode/reconstruct` in `pillar_udp.rs` realises a simpler first step, *not* yet the MDS target: M *identical* XOR-parity shards (each the XOR of all K data shards), which recover *exactly one* lost data shard—the extra parity copies only guard the parity itself against loss, so the test suite asserts "one parity cannot recover two holes." Its exact residual is therefore

$$P^{\text{xor}}_{\text{fail}}(K, M, p) = 1 - (1-p)^K - Kp(1-p)^{K-1}\big(1 - p^M\big), \qquad (5)$$

which for K=8 sits near 0.19 at p=0.1 *independent of* $M \geq 2$, against the MDS target's 4.3×10^{-3} at M=4: a single-erasure code cannot survive two concurrent data-shard losses however much parity it carries. Generalising the coding kernel to a true MDS code (Reed–Solomon / RaptorQ) is the specified bulk-regime work, and Eq. (4) is the residual that upgrade unlocks; every shard already carries a per-payload CID and a protocol-version stamp, so only the kernel changes.

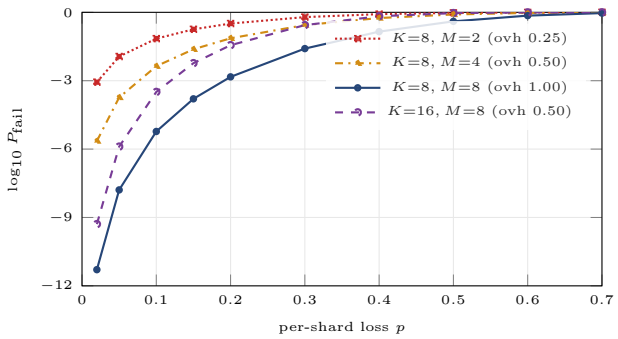

Figure 2: MDS (any-K-of-N) erasure residual loss (Eq. 4); the bulk-regime design target. Parity rate M/K sets the overhead; a larger block (K=16, M=8) beats a smaller one at the same rate by averaging loss over more shards. Lower is better; log scale.

p	0.05	0.10	0.20	0.30	0.40	0.50
$M^\star$	7	9	15	21	28	38
overhead $M^\star/K$	0.88	1.12	1.88	2.62	3.50	4.75

Table 2: Minimum parity $M^\star$ for an MDS code, K=8, ε=10^{-6} (Eq. 4). Beyond ∼30% loss even erasure overhead grows past 1×; a larger K (Fig. 2) is the cheaper lever there.

6 Adaptive allowance control

The allowance is not static: the sender measures loss and re-derives it. Let $\hat{p}_t$ be an exponentially weighted estimate of per-path loss updated per acknowledgement/CID-arrival observation $\ell_t \in \{0, 1\}$:

$$\hat{p}_t = (1-\alpha)\hat{p}_{t-1} + \alpha\,\ell_t, \qquad 0 < \alpha \ll 1, \qquad (6)$$

and set $W_t = \text{clamp}\big(W^\star(\hat{p}_t, \varepsilon),\ 1,\ W_{\max}\big)$ (or the parity analogue for bulk). Three properties make this stable where a window-based controller is not:

1. **Monotone, memoryless map.** $W^\star(\cdot, \varepsilon)$ is a fixed monotone step function of $\hat{p}_t$; the only state is the EWMA, a contraction of bounded inputs, so W_t cannot diverge or oscillate absent boundary chatter.
2. **Hysteresis at the steps.** $W^\star$ jumps at discrete $\hat{p}$ thresholds; we only change W when $\hat{p}_t$ crosses a threshold by a margin δ, eliminating flapping when $\hat{p}$ sits on a boundary.
3. **No positive feedback.** The control target is *loss*, and— crucially—redundancy is added only for *link* loss, not congestion (§7). On dispersed independent paths, raising W does not raise the $\hat{p}$ that drove it, so the loop has no self-reinforcing term. This is

the structural difference from TCP/QUIC, whose control variable (the window) directly drives the congestion signal it reacts to.

$W_{\max}$ is the connection's declared allowance ceiling; the controller may request up to it but never beyond, so a misbehaving loss estimate can never unbound fan-out.

7 Congestion safety

Redundancy is dangerous in exactly one regime: when the loss it is reacting to is *congestion* rather than link degradation. Adding copies to a congested bottleneck worsens the congestion—the classic collapse. pillar-UDP stays safe by three composable mechanisms.

1. Discriminate link loss from congestion. Congestion raises queueing delay *and* loss together; link loss raises loss with roughly flat one-way delay. The sender tracks a one-way delay gradient (the signal LEDBAT [11] and delay-based controllers use) and attributes to the link-loss component only the loss that is *not* accompanied by a rising delay gradient. Redundancy scales with the link-loss component; when the delay gradient indicates congestion, the controller *reduces* offered rate (and W) exactly like a well-behaved flow, rather than adding copies.

2. Marginal-gain stop. Even absent an explicit ε, the marginal delivery gain of the W-th copy is $\partial_W(1-p^W) = p^W \ln(1/p)$, which falls geometrically: for p=0.3 it is $0.36, 0.11, 0.033, 0.0098, \ldots$ for W=1, 2, 3, 4. The controller stops increasing W once the marginal delivered-probability per extra copy falls below a threshold τ, giving a natural cap that no target can push past on a good path.

3. Aggregate egress bound. Redundancy is admitted against capacity. For a node with egress capacity C carrying flows f at rates λ_f with mean message size S_f and allowance W_f, the local admission invariant is

$$\sum_f \lambda_f \, W_f \, S_f \;\le\; C. \tag{7}$$

Each node enforces (7) on its own egress; because redundant copies are *dispersed* across nodes and paths, any single bottleneck link carries only a fraction of a flow's fan-out, so the bound binds on aggregate egress, not on any one path. Together these ensure the adaptive allowance can raise reliability without ever becoming a congestion source: it responds to the loss it can fix (the link) and stands down on the loss it would worsen (congestion).

field	bytes	meaning
flags	1	version (3b), type (2b: spray/forward/reply/ack), effect-class (1b: convergent/exclusive), has-shard (1b), has-RR-token (1b)
CID	34	content-address multihash `<code><len><digest>` (SHA2-256 today, $0x12\,0x20$+32B; algorithm-agile); dedup + integrity key
allowance	1	mode bit + 7-bit value: copy count $W \in [1, 127]$, or parity rate M/K in 1/32 units
hop TTL	1	remaining forward budget; forwarding stops at 0
shard desc.	8†	$K(1)$, $M(1)$, shard index(2), payload length(4)
RR token	16†	return-routability cookie (keyed MAC of client address); anti-amplification

Table 3: pillar-UDP *specified* header. †present only when the corresponding flag is set. Base header (small message, no token) is 37 bytes; with shard descriptor 45; with RR token +16. (The live transport today stamps each shard's on-wire bytes with the 2-byte protocol version followed by the payload and verifies the CID per shard; the full header above is the specified encoding of the modelled datagram fields.)

8 Header wire format

Every datagram—spray copy, forward, reply, or acknowledgement—carries the header below, which binds it to its logical message and states the connection's redundancy contract up front. Fields are fixed-width and big-endian; optional fields are gated by flag bits so the common small-message header stays compact.

Framing and MTU. The header and payload ride as the application framing inside the pillar-UDP transport's Noise-encrypted, yamux-muxed stream ("Route A", the libp2p Noise+yamux upgrade the transport reuses). Datagrams are sized to the conservative IPv6 minimum path MTU: a small-message datagram keeps header + payload $\le$ 1200 bytes, and each bulk shard is sized so header + shard $\le$ 1200, so neither fragments. The header is therefore a fixed 37–61-byte prefix; for a 1 KB message its overhead is 3.5%, dwarfed by the $\times W$ payload cost, so the redundancy allowance, not the header, dominates on-wire bytes—as intended.

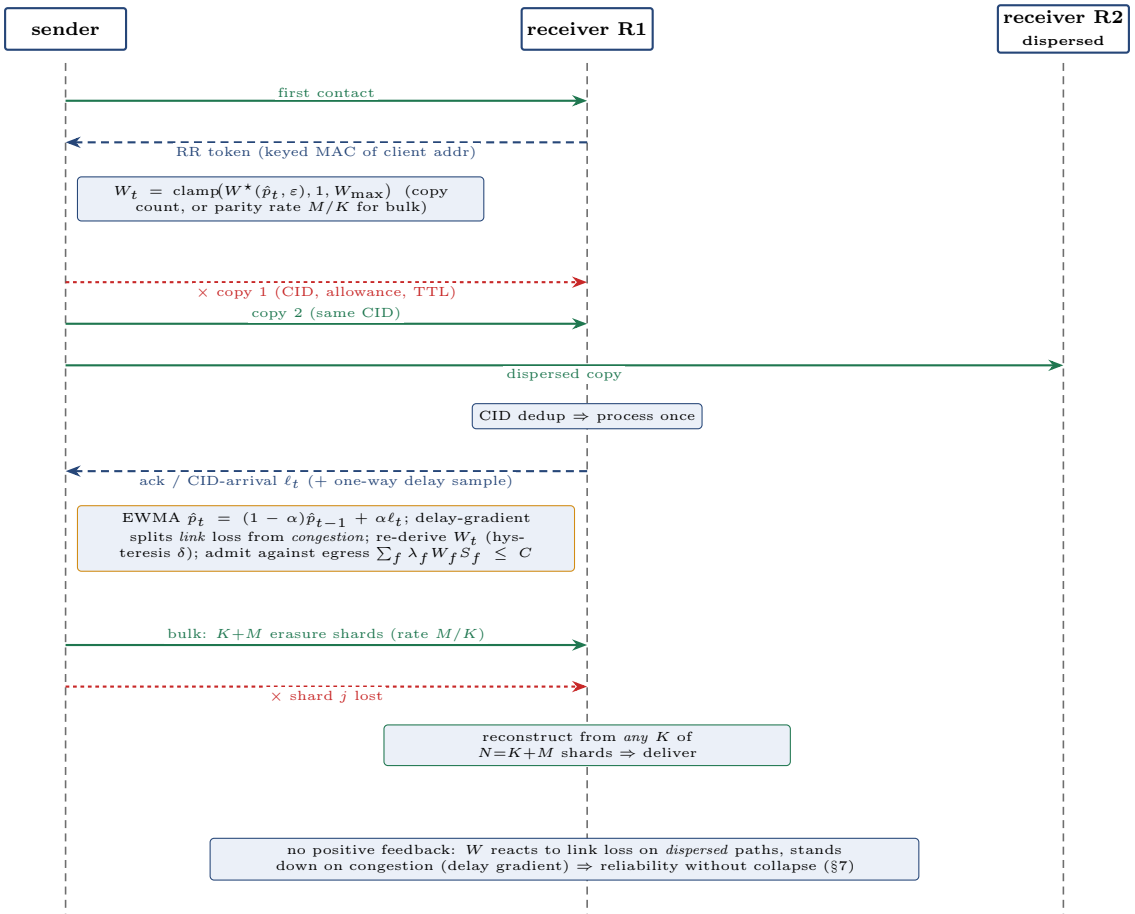

Figure 3: The redundancy control loop, in time. A sender establishes return-routability (RR token, anti-amplification), then emits an allowance-bounded set of CID-identical copies over dispersed paths; a lost copy costs nothing because a surviving copy carries the same CID and the receiver processes it once. The receiver's acknowledgement (a CID-arrival observation plus a one-way delay sample) feeds the EWMA loss estimate $\hat{p}_t$, from which the sender re-derives the allowance W_t with hysteresis—raising redundancy for *link* loss but standing down when the delay gradient indicates *congestion* (§6, §7), and always admitting against the egress bound (7). For bulk transfer the copies become an M-parity erasure code and the receiver reconstructs from any K of the N shards (§5). Because redundancy on dispersed independent paths does not raise the loss that drove it, the loop has no self-reinforcing term—the structural difference from a window-based controller.

Worked examples. A small idempotent spray copy with W=4, TTL=6, no shard, no token:

```
20 12 20 <32-byte SHA-256 digest> 04 06 <payload>
```

(`0x20`= version 1, type spray, convergent; the CID is the 34-byte multihash led by `12 20`). A bulk shard from a K=8, M=4 block, index 3, of a 1,048,576-byte payload:

```
22 12 20 <digest> 90 06 08 04 0003 00100000 <shard>
```

(`0x22` sets has-shard; `0x90` is parity mode at rate 16/32=0.5; the descriptor carries K=8, M=4, idx=3, len=0x100000).

9 Relation to the verified models

Every header field corresponds to an invariant already machine-checked in the primary paper's TLA$^+$ specifications, so the wire format is an encoding of proven mechanics, not new untested behaviour: the CID is the dedup key behind `ExactlyOnce`; the allowance bounds fan-out behind `SprayWidthBounded`/`BoundedTotalDatagrams`; the hop TTL is the forwarding-termination budget behind `NoForwardingLoops`; the effect-class bit is the idempotent/exclusive split routing exclusive processing to the coordination-core lease holder; and the RR token is the return-routability proof behind `AntiAmplificationBound` [1]. This paper adds the *quantities*—$W^\star$, $M^\star$, the ρ model, the control loop, and the safety envelope—that the qualitative proofs deliberately abstract away.

10 Related work

The loss-cliff that motivates the protocol is documented by Head [2]; the idea of surviving loss with redundant, source-dispersed copies rather than retransmission descends from real-time game networking [3]. Bursty loss is classically modelled by the Gilbert–Elliott two-state channel [4, 5], which we use to motivate the correlation estimand. Erasure coding for reliable transport spans Reed–Solomon codes [6], fountain codes such as RaptorQ [7], and on-the-fly coding such as Tetrys [8]; pillar-UDP's bulk regime is a systematic block code whose MDS target is selected by (4). QUIC [9] and its loss/congestion control [10] exemplify the single-path, window-throttled posture we contrast against; LEDBAT [11] supplies the delay-gradient discriminator we borrow for congestion safety; and MPTCP [12] shares the multipath aim but retains per-subflow window control rather than redundancy. The correctness scaffolding is TLA$^+$ [13].

11 Limitations and future work

The analysis is exact but rests on two modelling choices whose empirical calibration is future work: the marginal-p Bernoulli-with-correlation loss model (§4), whose single parameter ρ must be measured across real dispersed pillar paths by the methodology given rather than assumed; and the independence *within* a block for erasure (§5), which the same correlation study should validate for shards as it does for copies. The delay-gradient discriminator (§7) inherits the known difficulty of delay-based congestion signals under reordering and needs the same live validation. None of these affects the proven safety and liveness guarantees, which hold for any loss pattern; they affect only how tightly the allowance can be tuned. We report no measured numbers here precisely because we have not run that study; the framework is built so its results drop in without changing any equation.

12 Conclusion

Setting pillar-UDP's redundancy is a bounded optimisation: choose the smallest allowance—copy count $W^\star$ or parity $M^\star$—that meets the connection's residual target at the measured loss, adapt it as loss changes, and never let the aggregate exceed egress. The reliability it buys is real and super-linear, but only to the extent the dispersed paths are uncorrelated: Eq. (3) makes that dependence exact and turns "spray from diverse IPs" from folklore into the quantitative lever it is. With the wire format fixed and the control loop kept congestion-safe by construction, the allowance becomes a single, well-understood knob spanning a clean link (where $W{=}1$ costs nothing) to a hostile one (where redundancy delivers what window backoff cannot).

References

[1] pillar engineering, *pillar-UDP: Clustered Multipath Congestion Avoidance and Redundant Delivery over Lossy Networks*, 2026 (companion paper).

[2] G. Head, *Reliable Delivery Over Horrible Networks*, 2026. `https://blog.graysonhead.net/posts/reliable-delivery/`

[3] M. Frohnmayer and T. Gift, *The TRIBES Engine Networking Model*, 2000.

[4] E. N. Gilbert, "Capacity of a burst-noise channel," *Bell System Technical Journal*, 39(5):1253–1265, 1960.

[5] E. O. Elliott, "Estimates of error rates for codes on burst-noise channels," *Bell System Technical Journal*, 42(5):1977–1997, 1963.

[6] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *J. SIAM*, 8(2):300–304, 1960.

[7] M. Luby et al., *RaptorQ Forward Error Correction Scheme for Object Delivery*, RFC 6330, IETF, 2011.

[8] J. Detchart, E. Lochin, J. Lacan, V. Roca, *Tetrys: An On-the-Fly Network Coding Protocol*, RFC 9407, IETF, 2023.

[9] J. Iyengar and M. Thomson (eds.), *QUIC: A UDP-Based Multiplexed and Secure Transport*, RFC 9000, IETF, 2021.

[10] J. Iyengar and I. Swett (eds.), *QUIC Loss Detection and Congestion Control*, RFC 9002, IETF, 2021.

[11] S. Shalunov et al., *Low Extra Delay Background Transport (LEDBAT)*, RFC 6817, IETF, 2012.

[12] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, C. Paasch, *TCP Extensions for Multipath Operation with Multiple Addresses*, RFC 8684, IETF, 2020.

[13] L. Lamport, *Specifying Systems*, Addison-Wesley, 2002.